PYTHON PROGRAMMING – ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT

Student Name: Koduru Naga Deepika
Roll Number: 26000091
Branch: CSE
Section: 2
Course: Python Programming

Case Study 1: University Scholarship Allocation

1. Problem Statement

Check whether a student is eligible for a scholarship based on marks, attendance, and family income.

2. Objectives

To check scholarship eligibility using conditions.

3. Assumptions

Minimum marks are 75%, attendance is 75%, and annual family income must not exceed ₹2,00,000.

4. Algorithm

1. Start.
2. Read marks, attendance, and income.
3. Check all eligibility conditions.
4. Display the result.
5. Stop.

5. Flowchart

Start → Input details → Check eligibility → Display result → Stop.

6. Pseudocode

```
READ marks, attendance, income
IF marks >= 75 AND attendance >= 75 AND income <= 200000
   PRINT "Eligible"
ELSE
   PRINT "Not Eligible"
```

END IF

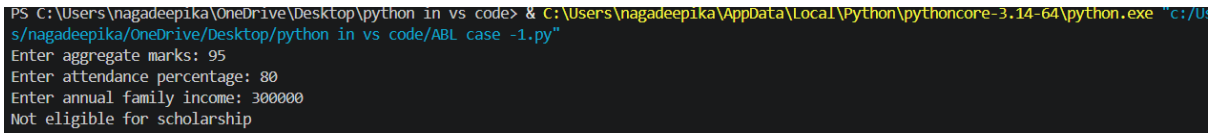
7.Python Source Code

```
marks = int(input("Enter marks: "))
attendance = int(input("Enter attendance: "))
income = int(input("Enter annual income: "))
if marks >= 75 and attendance >= 75 and income <= 200000:
  print("Eligible for scholarship")
else:
  print("Not eligible")
```

8. Test Cases

Marks	Attendance	Income	Output
85	80	150000	Eligible
70	80	150000	Not eligible

9. Output



10. Conclusion

The program checks scholarship eligibility using conditional statements.

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Calculate an electricity bill based on units consumed and different unit rates.

2. Objectives

To calculate electricity charges using conditions.

3. Assumptions

First 100 units cost ₹2 per unit, next 100 cost ₹3, and units above 200 cost ₹5.

4. Algorithm

1. Start.
2. Read units.
3. Calculate the bill according to the unit range.
4. Display the bill.
5. Stop.

5. Flowchart

Start → Input units → Calculate bill → Display bill → Stop.

6. Pseudocode

```
READ units
IF units <= 100
   bill = units * 2
ELSE IF units <= 200
   bill = 200 + (units - 100) * 3
ELSE
   bill = 500 + (units - 200) * 5
END IF
PRINT bill
```

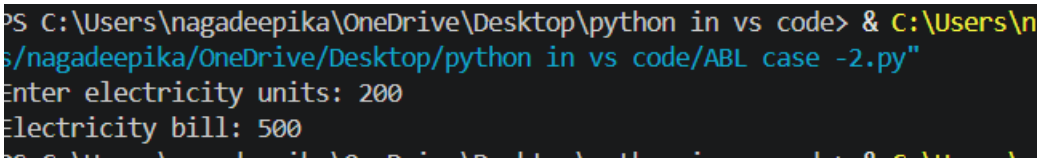
7. Python Source Code

```
units = int(input("Enter units: "))
if units <= 100:
   bill = units * 2
elif units <= 200:
   bill = 200 + (units - 100) * 3
else:
   bill = 500 + (units - 200) * 5
print("Electricity bill:", bill)
```

8. Test Cases

Units	Expected Bill
80	₹160
150	₹350
250	₹750

9. Output



10. Conclusion

The program calculates electricity charges using progressive rates.

Case Study 3: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate an employee's HRA, DA, deduction, and net salary.

2. Objectives

To calculate salary components using arithmetic operations.

3. Assumptions

HRA is 10%, DA is 5%, and deduction is 8% of basic salary.

4. Algorithm

1. Start.
2. Read basic salary.
3. Calculate HRA, DA, and deduction.
4. Calculate net salary.
5. Display the results.
6. Stop.

5. Flowchart

Start → Input salary → Calculate components → Display net salary → Stop.

6. Pseudocode

READ salary

hra = salary * 0.10

da = salary * 0.05

deduction = salary * 0.08

net = salary + hra + da - deduction

PRINT net

7.Python Source Code

```
salary = float(input("Enter basic salary: "))

hra = salary * 0.10

da = salary * 0.05

deduction = salary * 0.08

net = salary + hra + da - deduction

print("HRA:", hra)

print("DA:", da)

print("Deduction:", deduction)

print("Net salary:", net)
```

8. Test Cases

Basic Salary	Net Salary
₹10000	₹10700
₹20000	₹21400

9. Output

```
s/nagadeepika/OneDrive/Desktop/python in vs code/ABL case -3.py"
Enter basic salary: 30000
HRA: 3000.0
DA: 1500.0
Deduction: 2400.0
Net salary: 32100.0
```

10. Conclusion

The program calculates salary components and net salary.

Case Study 4: Retail Billing with Multiple Discount Rules

1. Problem Statement

Calculate a customer's discount and final purchase amount.

2. Objectives

To apply discount rules using conditional statements.

3. Assumptions

Purchases of ₹5000 or more receive 20% discount. Purchases of ₹2000 or more receive 10%; other purchases receive no discount.

4. Algorithm

1. Start.
2. Read purchase amount.
3. Calculate discount.
4. Subtract discount from amount.
5. Display final bill.
6. Stop.

5. Flowchart

Start → Input amount → Check discount → Calculate total → Display bill → Stop.

6. Pseudocode

```
READ amount
IF amount >= 5000
   discount = amount * 0.20
ELSE IF amount >= 2000
   discount = amount * 0.10
ELSE
   discount = 0
END IF
PRINT amount - discount
```

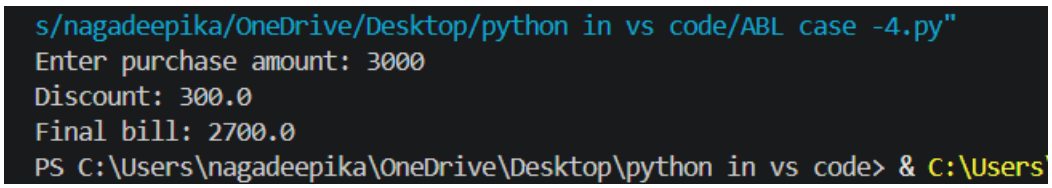
7.Python Source Code

```
amount = float(input("Enter purchase amount: "))
if amount >= 5000:
   discount = amount * 0.20
elif amount >= 2000:
   discount = amount * 0.10
else:
   discount = 0
print("Discount:", discount)
print("Final bill:", amount - discount)
```

8. Test Cases

Amount	Discount	Final Bill
₹1000	₹0	₹1000
₹3000	₹300	₹2700
₹6000	₹1200	₹4800

9. Output



10. Conclusion

The program calculates discounts and the final retail bill.

Case Study 5: Campus Event Financial Planning

1. Problem Statement

Calculate the income, expenses, and balance for a campus event.

2. Objectives

To analyse an event's budget using basic calculations.

3. Assumptions

Income is calculated using participant count and fee per participant.

4. Algorithm

1. Start.
2. Read participants, fee, and expenses.
3. Calculate total income.
4. Calculate balance.
5. Display the result.
6. Stop.

5. Flowchart

Start → Input details → Calculate income and balance → Display result → Stop.

6. Pseudocode

READ participants, fee, expenses

income = participants * fee

balance = income - expenses

PRINT income, expenses, balance

IF balance >= 0

  PRINT "Within budget"

ELSE

  PRINT "Loss"

END IF

7.Python Source Code

```
participants = int(input("Enter participants: "))
fee = float(input("Enter fee per participant: "))
expenses = float(input("Enter expenses: "))
income = participants * fee
balance = income - expenses
print("Income:", income)
print("Expenses:", expenses)
```

```
print("Balance:", balance)

if balance >= 0:

  print("Within budget")

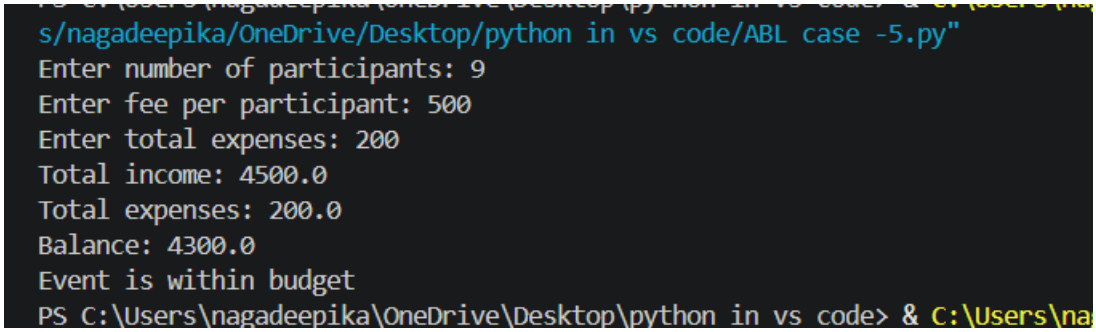
else:

  print("Loss")
```

8. Test Cases

Participants	Fee	Expenses	Balance
100	₹200	₹15000	₹5000
50	₹100	₹6000	−₹1000

9. Output



10. Conclusion

The program checks whether a campus event has a surplus or loss.

Case Study 6: Examination Result and Grace-Mark Validation

1. Problem Statement

Determine a student's result based on examination marks.

2. Objectives

To classify examination results using conditions.

3. Assumptions

Marks of 40 or above indicate a pass. Marks from 35 to 39 indicate that grace marks may be required, subject to college rules.

4. Algorithm

1. Start.
2. Read marks.
3. Check the marks range.
4. Display the result.
5. Stop.

5. Flowchart

Start → Input marks → Check range → Display result → Stop.

6. Pseudocode

```
READ marks
IF marks >= 40
   PRINT "Pass"
ELSE IF marks >= 35
   PRINT "Grace marks may be required"
ELSE
   PRINT "Fail"
END IF
```

7.Python Source Code

```
marks = int(input("Enter marks: "))
if marks >= 40:
   print("Pass")
elif marks >= 35:
   print("Grace marks may be required")
else:
   print("Fail")
```

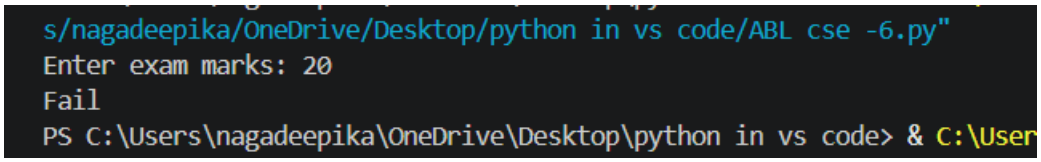
8. Test Cases

Marks	Output
75	Pass

37 Grace marks may be required

25 Fail

9. Output



10. Conclusion

The program classifies results using conditional statements.

Case Study 7: Attendance Recovery Computation

1. Problem Statement

Calculate attendance percentage and determine how many additional classes are needed to reach 75% attendance.

2. Objectives

To calculate attendance and plan attendance recovery.

3. Assumptions

The minimum required attendance is 75%, and every additional class is attended.

4. Algorithm

1. Start.
2. Read total classes and attended classes.
3. Calculate attendance percentage.
4. If below 75%, calculate the required additional classes.
5. Display the result.
6. Stop.

5. Flowchart

Start → Input classes → Calculate percentage → Check 75% → Display result → Stop.

6. Pseudocode

READ total, attended

```
attendance = attended * 100 / total
PRINT attendance
IF attendance < 75
   extra = (0.75 * total - attended) / 0.25
   PRINT extra classes required
ELSE
   PRINT "Attendance sufficient"
END IF
```

7.Python Source Code

```
total = int(input("Enter total classes: "))
attended = int(input("Enter attended classes: "))
if total <= 0 or attended < 0 or attended > total:
   print("Invalid input")
else:
   attendance = attended * 100 / total
   print("Attendance:", attendance, "%")
   if attendance >= 75:
      print("Attendance sufficient")
   else:
      extra = (0.75 * total - attended) / 0.25
      print("Extra classes needed:", int(extra) + 1)
```

8. Test Cases

Total	Attended	Output
100	80	Sufficient
100	60	60 extra classes

9. Output

```
s/nagadeepika/OneDrive/Desktop/python in vs code/ABL case -7.py"
Enter total classes: 200
Enter classes attended: 170
Attendance: 85.0 %
Attendance is sufficient
PS C:\Users\nagadeepika\OneDrive\Desktop\python in vs code> & C:\User
```

10. Conclusion

The program calculates attendance and estimates the additional classes needed.

Case Study 8: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate ATM operations such as checking balance, depositing money, and withdrawing money.

2. Objectives

To use loops and conditions for basic banking transactions.

3. Assumptions

The initial balance is ₹5000. Withdrawals cannot exceed the available balance.

4. Algorithm

1. Start with an initial balance.
2. Display the ATM menu.
3. Read the user's choice.
4. Perform the selected transaction.
5. Repeat until exit.
6. Stop.

5. Flowchart

Start → Display menu → Read choice → Perform transaction → Repeat or Exit.

6. Pseudocode

balance = 5000

REPEAT

DISPLAY menu

```
    READ choice
    IF choice = 1
        PRINT balance
    ELSE IF choice = 2
        READ amount
        ADD amount to balance
    ELSE IF choice = 3
        READ amount
        IF amount <= balance
            SUBTRACT amount
        ELSE
            PRINT "Insufficient balance"
        END IF
    ELSE IF choice = 4
        EXIT
    END IF
UNTIL exit
```

7.Python Source Code

```
balance = 5000
while True:
    print("1. Check Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        print("Balance:", balance)
    elif choice == 2:
```

```
        amount = int(input("Enter deposit: "))
        if amount > 0:
            balance += amount
            print("Deposit successful")
        else:
            print("Invalid amount")
    elif choice == 3:
        amount = int(input("Enter withdrawal: "))
        if 0 < amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Invalid amount or insufficient balance")
    elif choice == 4:
        break
    else:
        print("Invalid choice")
```

8. Test Cases

Operation	Amount	Expected Result
Check balance	—	₹5000
Deposit	₹1000	₹6000
Withdraw	₹2000	₹4000

9. Output

```
s/nagadeepika/OneDrive/Desktop/python in vs code/ABL case -8.py"
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter your choice: 1
Balance: 5000
```

10. Conclusion

The program simulates basic ATM operations using loops and conditions.

Case Study 9: Warehouse Inventory Reconciliation

1. Problem Statement

Display item stock and update inventory after a sale.

2. Objectives

To manage inventory using lists and list operations.

3. Assumptions

The warehouse stores three items: Pen, Book, and Bag.

4. Algorithm

1. Start.
2. Create item and stock lists.
3. Display the stock.
4. Read the sold item and quantity.
5. Check stock and update it.
6. Stop.

5. Flowchart

Start → Create lists → Display stock → Read sale → Check availability → Update stock → Stop.

6. Pseudocode

```
items = ["Pen", "Book", "Bag"]

stock = [50, 30, 20]

DISPLAY items and stock

READ item, quantity

IF item exists

  FIND item index

  IF quantity <= stock[index]
```

```
    REDUCE stock[index]
  ELSE
    PRINT "Insufficient stock"
  END IF
ELSE
  PRINT "Item not found"
END IF
```

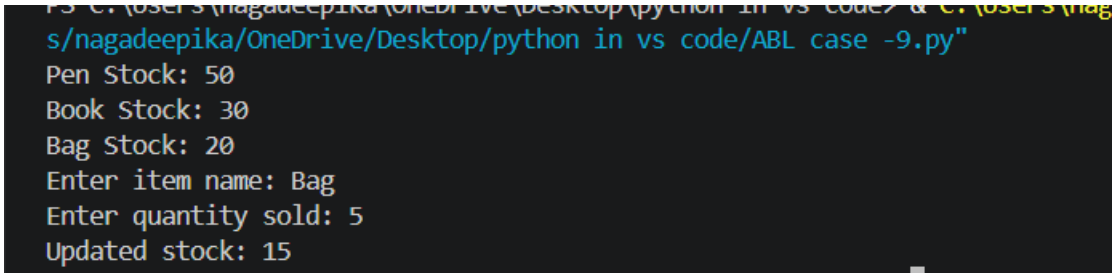
7.Python Source Code

```
items = ["Pen", "Book", "Bag"]
stock = [50, 30, 20]
for i in range(len(items)):
  print(items[i], stock[i])
item = input("Enter item name: ")
quantity = int(input("Enter quantity sold: "))
if item in items:
  i = items.index(item)
  if 0 < quantity <= stock[i]:
    stock[i] -= quantity
    print("Remaining stock:", stock[i])
  else:
    print("Invalid quantity or insufficient stock")
else:
  print("Item not found")
```

8. Test Cases

Item	Initial Stock	Sold	Remaining
Pen	50	10	40
Book	30	5	25

9. Output



10. Conclusion

The program updates warehouse stock using lists and conditions.

Case Study 10: Advanced Numeric Pattern Generator

1. Problem Statement

Generate a number pattern based on the number of rows entered by the user.

2. Objectives

To generate patterns using nested loops.

3. Assumptions

The user enters a positive number of rows.

4. Algorithm

1. Start.
2. Read the number of rows.
3. Use an outer loop for rows.
4. Use an inner loop to print numbers.
5. Stop.

5. Flowchart

Start → Input rows → Outer loop → Inner loop → Print pattern → Stop.

6. Pseudocode

READ n

FOR i from 1 to n

  FOR j from 1 to i

PRINT j

END FOR

PRINT new line

END FOR

7.Python Source Code

```
n = int(input("Enter number of rows: "))
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

8. Test Cases

Input	Expected Pattern
3	1 / 1 2 / 1 2 3
4	1 / 1 2 / 1 2 3 / 1 2 3 4

9. Output

```
s/nagadeepika/OneDrive/Desktop/python in vs code/ABL case -10.p
Enter number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
PS C:\Users\nagadeepika\OneDrive\Desktop\python in vs code>
```

10. Conclusion

The program generates numeric patterns using nested loops.